

# GitHub Actions Runners & Runtime

How jobs flow from YAML to execution — runner lifecycle, ARC, scaling, and internals

---

## TOPICS COVERED

- › Hosted Runner Architecture
- › Runner Lifecycle & Registration
- › Job Scheduling Algorithm
- › Ephemeral Runners
- › ARC (Actions Runner Controller)
- › Autoscaling Patterns
- › YAML → Execution Flow
- › Dependency Graph
- › Artifact & Cache Infrastructure
- › Self-Hosted Runner Architecture
- › Authentication & Heartbeat
- › Runner Labels & Groups
- › Container Runners
- › Kubernetes Runners
- › High Availability
- › Workflow Parser & Job Planner
- › Secrets Injection
- › Cost Optimization

# PART 4 — GitHub Actions Runner Deep Dive

## 4.1 Hosted Runners

GitHub-hosted runners are ephemeral VMs provisioned on demand by GitHub's infrastructure. Each job receives a fresh VM that is terminated after the job completes. This guarantees isolation but means no persistent state between jobs.

### Runner Specifications

| Runner Label                  | OS                  | CPU         | RAM    | Storage    | Notes               |
|-------------------------------|---------------------|-------------|--------|------------|---------------------|
| ubuntu-latest / ubuntu-24.04  | Ubuntu 24.04        | 4 vCPU      | 16 GB  | 14 GB SSD  | Default; free tier  |
| ubuntu-22.04                  | Ubuntu 22.04        | 4 vCPU      | 16 GB  | 14 GB SSD  | LTS stable          |
| windows-latest / windows-2022 | Windows Server 2022 | 2 vCPU      | 16 GB  | 14 GB SSD  | PowerShell/cmd      |
| macos-latest / macos-14       | macOS 14 (Sonoma)   | 3 vCPU (M1) | 16 GB  | 14 GB SSD  | 10x billing         |
| ubuntu-latest-16-cores        | Ubuntu 24.04        | 16 vCPU     | 64 GB  | 150 GB SSD | Larger runner; paid |
| ubuntu-latest-64-cores        | Ubuntu 24.04        | 64 vCPU     | 256 GB | 2 TB SSD   | Larger runner; paid |

■ *macOS runners are billed at 10x the Linux rate due to Apple hardware licensing restrictions. Always evaluate whether macOS is truly required — many build/test tasks work on Linux with cross-compilation.*

### Hosted Runner Software

GitHub-hosted runners come pre-installed with hundreds of tools. The full list is maintained in the runner-images repository at [github.com/actions/runner-images](https://github.com/actions/runner-images). This includes: Docker, Node.js (multiple versions), Python (multiple versions), Java (Temurin), .NET SDK, Go, Ruby, AWS CLI, Azure CLI, Google Cloud SDK, kubectl, Helm, Terraform, Packer, and many more.

```
# See exactly what's installed on a runner:
- name: Dump tool versions
  run: |
    echo "=== OS ===" && lsb_release -a
    echo "=== Docker ===" && docker --version
    echo "=== Node ===" && node --version
    echo "=== Python ===" && python3 --version
    echo "=== Java ===" && java -version 2>&1
    echo "=== Go ===" && go version
    echo "=== kubectl ===" && kubectl version --client
    echo "=== Full list ===" && dpkg -l | grep -E "^ii" | wc -l
```

## 4.2 Self-Hosted Runner Architecture

Self-hosted runners run on infrastructure you control (bare metal, VMs, Kubernetes, or containers). They communicate outbound to GitHub's API over HTTPS/443 — no inbound firewall rules required.

### Runner Registration Flow

```
# 1. Generate a registration token (expires in 1 hour):
curl -L -X POST \
  -H "Authorization: Bearer $GITHUB_TOKEN" \
  -H "Accept: application/vnd.github+json" \
  https://api.github.com/repos/ORG/REPO/actions/runners/registration-token

# 2. Download and configure the runner:
mkdir actions-runner && cd actions-runner
curl -O -L https://github.com/actions/runner/releases/download/v2.317.0/actions-runner-linux-x64-2.317.0.tar.gz
tar xzf ./actions-runner-linux-x64-2.317.0.tar.gz

./config.sh \
  --url https://github.com/ORG/REPO \
  --token REGISTRATION_TOKEN \
  --name "prod-runner-01" \
  --labels "self-hosted,linux,x64,production,gpu" \
  --runnergroup "production-runners" \
  --ephemeral # Delete runner after single job (security best practice)

# 3. Start (as service):
sudo ./svc.sh install
sudo ./svc.sh start
```

## Runner Communication Protocol

The runner uses a long-polling protocol over HTTPS. It is NOT a persistent WebSocket — it polls the Actions service at intervals.

```
# Runner → GitHub polling loop:
1. POST /runner/session → Get session token (JWT)
2. GET /runner/message?sessionId=... (30s long poll)
   → If job available: returns JobMessage with credentials
   → If timeout: repeat
3. POST /runner/session (renew credentials)
4. Execute job steps
5. Stream logs via WebSocket to log service
6. POST /runner/complete → Report job result
```

■ *The runner's outbound HTTPS communication means it works through corporate proxies. Configure `HTTPS_PROXY`, `HTTP_PROXY`, and `NO_PROXY` environment variables in the runner service environment.*

## Security: Runner Isolation

| Isolation level        | Mechanism                  | Security                   | Performance          |
|------------------------|----------------------------|----------------------------|----------------------|
| Hosted VM              | Separate Azure VM per job  | Excellent                  | ~30-60s startup      |
| Self-hosted ephemeral  | Fresh container/VM per job | Good (clean state)         | Configurable         |
| Self-hosted persistent | Shared runner state        | Poor (history leaks)       | Fast (no cold start) |
| ARC container          | Kubernetes pod per job     | Good (namespace isolation) | ~5-10s startup       |
| ARC VM (KubeVirt)      | VM per job on K8s          | Excellent                  | ~30s startup         |

■ *NEVER use persistent self-hosted runners for untrusted code (public repos or external PRs). Malicious PR code can read `GITHUB_TOKEN`, access previous job artifacts, or access mounted credentials. Always use ephemeral runners.*

## 4.3 ARC — Actions Runner Controller

ARC is a Kubernetes operator that manages the lifecycle of self-hosted GitHub Actions runners on Kubernetes. It replaces the need for custom scripts to register/deregister runners and provides auto-scaling based on the Actions

job queue.

## ARC Architecture

```
# ARC components:
# - controller-manager: Kubernetes controller watching GitHub API
# - RunnerDeployment: Declares desired runner pods
# - RunnerSet: Newer, more efficient implementation
# - HorizontalRunnerAutoscaler: Scales based on queue depth

# Install ARC with Helm:
helm repo add actions-runner-controller \
  https://actions-runner-controller.github.io/actions-runner-controller
helm install arc \
  --namespace arc-systems \
  --create-namespace \
  actions-runner-controller/actions-runner-controller \
  --set authSecret.create=true \
  --set authSecret.github_token="$GITHUB_TOKEN"

# RunnerSet definition:
apiVersion: actions.github.com/v1alpha1
kind: RunnerSet
metadata:
  name: production-runners
  namespace: arc-runners
spec:
  replicas: 3
  githubConfigUrl: https://github.com/myorg
  githubConfigSecret: controller-manager-secret
  maxReplicas: 20
  minReplicas: 1
  containerMode:
    type: kubernetes # or "dind" for Docker-in-Docker
    kubernetesModeWorkVolumeClaim:
      accessModes: [ReadWriteOnce]
      storageClassName: fast-ssd
    resources:
      requests:
        storage: 10Gi
  template:
    spec:
      containers:
        - name: runner
          image: ghcr.io/actions/actions-runner:latest
          resources:
            requests:
              cpu: "500m"
              memory: "512Mi"
            limits:
              cpu: "4"
              memory: "8Gi"
      tolerations:
        - key: "runner"
          operator: "Equal"
          value: "true"
          effect: "NoSchedule"
      nodeSelector:
        runner-type: "ci-runner"

# HorizontalRunnerAutoscaler:
apiVersion: actions.github.com/v1alpha1
kind: HorizontalRunnerAutoscaler
metadata:
  name: arc-hra
spec:
  scaleTargetRef:
    name: production-runners
  minReplicas: 1
  maxReplicas: 20
  metrics:
    - type: TotalNumberOfQueuedAndInProgressWorkflowRuns
      repositoryNames:
        - myorg/api-service
        - myorg/frontend
```

## ARC Runner Modes

| Mode               | How it works                          | Docker support      | Best for                   |
|--------------------|---------------------------------------|---------------------|----------------------------|
| Kubernetes mode    | Steps run as K8s pods in workflow pod | Via kaniko/buildkit | Secure, regulated          |
| DinD mode          | Docker-in-Docker in runner container  | Full Docker daemon  | Flexibility, Docker builds |
| VM mode (KubeVirt) | Actual VMs scheduled by K8s           | Full Docker         | Maximum isolation          |

## 4.4 Artifact and Cache Infrastructure

### Artifacts

Actions artifacts are files uploaded during a workflow run and stored on GitHub's infrastructure for a configurable retention period (default 90 days, minimum 1 day).

```
# Upload artifacts:
- uses: actions/upload-artifact@v4
  with:
    name: build-output-${{ github.sha }}
    path: |
      dist/
      reports/coverage.xml
      reports/test-results.xml
    retention-days: 30
    compression-level: 6 # 0-9, default 6
    if-no-files-found: error # warn | ignore | error

# Download in subsequent job:
- uses: actions/download-artifact@v4
  with:
    name: build-output-${{ github.sha }}
    path: ./artifacts

# Download all artifacts from the run:
- uses: actions/download-artifact@v4
  with:
    path: ./all-artifacts
    pattern: build-output-*
    merge-multiple: true
```

### Caching

The actions/cache action stores and retrieves build caches using content-addressable keys. Cache entries are stored at the organization level and shared across repositories in the same org.

```
# Cache with restore keys fallback chain:
- uses: actions/cache@v4
  id: cache
  with:
    path: |
      ~/.cache/pip
      ~/.local/lib/python*/site-packages
    key: pip-${{ runner.os }}-${{ hashFiles('**/requirements.txt') }}
    restore-keys: |
      pip-${{ runner.os }}-
      pip-
    # key: exact match or miss
    # restore-keys: prefix match, most recent wins

# Check cache hit:
- if: steps.cache.outputs.cache-hit != 'true'
  run: pip install -r requirements.txt
# Cache eviction:
```

```
# - Entries expire after 7 days of non-access
# - Max cache size: 10 GB per repository
# - When limit is reached, oldest entries are evicted first
# - Cross-repo sharing within org requires same cache key

# Advanced: save cache even on failure:
- uses: actions/cache/save@v4
  if: always()
  with:
    path: ~/.cache/pip
    key: ${ steps.cache.outputs.cache-primary-key }
```

## Cache Strategy Patterns

| Language        | Paths to cache            | Key strategy                                         |
|-----------------|---------------------------|------------------------------------------------------|
| Node.js         | ~/.npm or node_modules    | OS + hashFiles('**/package-lock.json')               |
| Python (pip)    | ~/.cache/pip              | OS + Python version + hashFiles('requirements*.txt') |
| Python (poetry) | ~/.cache/pypoetry         | OS + hashFiles('poetry.lock')                        |
| Go              | ~/go/pkg/mod              | OS + hashFiles('go.sum')                             |
| Java (Maven)    | ~/.m2/repository          | OS + hashFiles('**/pom.xml')                         |
| Java (Gradle)   | ~/.gradle/caches          | OS + hashFiles('**/*.gradle*')                       |
| Rust (Cargo)    | ~/.cargo + target/        | OS + hashFiles('Cargo.lock')                         |
| Docker layers   | via BuildKit gha exporter | Automatic via docker/build-push-action               |

## 4.5 Cost Optimization

### Runner Minute Billing

GitHub Actions is billed per minute of runner usage. Free tier: 2,000 min/month (personal), 3,000 min/month (Team), unlimited (Enterprise). Paid rates:

| Runner type       | Rate        | Multiplier vs Linux |
|-------------------|-------------|---------------------|
| Linux (2-core)    | \$0.008/min | 1x                  |
| Windows (2-core)  | \$0.016/min | 2x                  |
| macOS (3-core M1) | \$0.08/min  | 10x                 |
| Linux 4-core      | \$0.016/min | 2x                  |
| Linux 8-core      | \$0.032/min | 4x                  |
| Linux 16-core     | \$0.064/min | 8x                  |
| Linux 64-core     | \$0.256/min | 32x                 |

### Cost Reduction Strategies

- **Path filtering:** Use `paths:` and `paths-ignore:` to skip workflows when only docs change
- **Concurrency cancel:** Cancel in-progress runs on PR pushes — saves minutes on superseded commits

- **Conditional jobs:** Use if: conditions to skip downstream jobs when upstream fails
- **Cache aggressively:** A warm cache can reduce a 10-minute build to 2 minutes
- **Self-hosted runners:** For high-volume orgs, self-hosted on ARM (Graviton) or spot instances is 5-10x cheaper
- **Merge queue:** Test only merged combinations rather than each PR independently
- **Larger runners for faster builds:** A 16-core runner at 4x cost that finishes in 1/4 the time is cost-neutral but faster

```
# Cost-efficient workflow pattern:
on:
  push:
    branches: [main]
    paths:
      - 'src/**'
      - 'tests/**'
      - 'package*.json'
      - '.github/workflows/**'

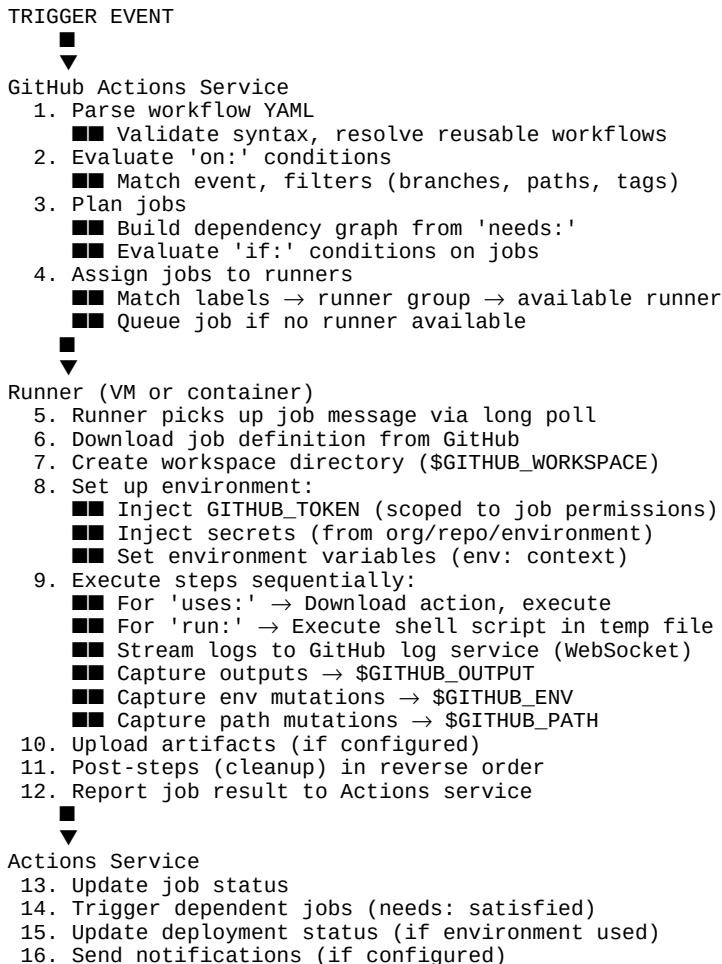
jobs:
  quick-check:
    runs-on: ubuntu-latest # Cheapest option
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0 # Only for git log/diff
      - name: Check if only docs changed
        id: changes
        run: |
          if git diff HEAD~1 --name-only | grep -qE '^(docs|*.md)'; then
            echo "has-code-changes=true" >> $GITHUB_OUTPUT
          fi

  expensive-build:
    needs: quick-check
    if: needs.quick-check.outputs.has-code-changes == 'true'
    runs-on: ubuntu-latest-16-cores # Larger runner for speed
```

## PART 5 — GitHub Actions Runtime

### 5.1 YAML to Execution: The Complete Flow

Understanding the internal execution flow helps diagnose issues and optimize workflow performance. Here is the complete pipeline:



### GITHUB\_TOKEN Scoping

Each job receives a unique GITHUB\_TOKEN with permissions scoped to that job's 'permissions:' block. The token is minted by GitHub's token service and automatically expires when the job ends.

```

# Token permissions (default depends on org settings):
permissions:
  actions: read           # Read workflow runs
  checks: write           # Create check runs (for test reporting)
  contents: write         # Push commits, create releases
  deployments: write      # Create deployment records
  id-token: write         # Request OIDC JWT
  issues: write           # Comment on issues
  packages: write         # Push to GHCR
  pages: write            # Deploy to GitHub Pages
  pull-requests: write    # Comment on PRs, approve
  security-events: write  # Upload SARIF results
  statuses: write         # Update commit statuses

```



```
# Principle of least privilege:
# Only grant what each job actually needs.
# Wrong (overly broad):
permissions: write-all

# Right:
permissions:
  contents: read
  packages: write
```

## Secrets Injection

Secrets are injected into the runner environment by the Actions service, not stored on the runner host. They are:

- Encrypted at rest using GitHub's key management service
  - Transmitted to the runner over a TLS-encrypted channel
  - Masked in logs — any output matching the secret value is replaced with \*\*\*
  - Available only to jobs that match the environment's protection rules
  - Scoped to the requesting repository (not leaked to forked repos)
- ```
# Secret masking – GitHub automatically masks secrets in logs.
# But watch for these masking bypass patterns:
- name: WRONG - logs base64-encoded secret
  run: echo "$SECRET" | base64 # Masked value, different encoding = leaked!

- name: RIGHT - add masks explicitly for derived values
  run: |
    ENCODED=$(echo "$SECRET" | base64)
    echo "::add-mask::$ENCODED" # Mask the derived value too
    echo "Using encoded secret..."
  env:
    SECRET: ${ secrets.API_KEY }
```

## 5.2 Log Streaming Architecture

Actions logs are streamed in real-time from the runner to GitHub's log infrastructure via a WebSocket connection. Logs are:

- Chunked into numbered, ordered packets
  - Compressed with gzip before transmission
  - Stored in Azure Blob Storage
  - Retained for 90 days by default (configurable 1-400 days)
  - Searchable via the GitHub web UI and available via API
- ```
# Structured log output using workflow commands:
- name: Structured logging demo
  run: |
    # Create a named log group (collapsible in UI):
    echo "::group::Installing dependencies"
    npm install
    echo "::endgroup::"

    # Set output variables:
    echo "version=$(node -p "require('./package.json').version")" >> $GITHUB_OUTPUT

    # Debug message (only visible when debug logging enabled):
    echo "::debug::Resolved package version: 1.2.3"

    # Notice (appears in annotations):
    echo "::notice file=src/app.js,line=42::Performance warning: consider caching"

    # Warning annotation:
    echo "::warning file=Dockerfile,line=5::Base image is out of date"

    # Error annotation:
    echo "::error file=src/auth.py,line=17::Missing input validation"
```

```
# Fail a step without exiting shell:  
echo "::error::Validation failed" && exit 1
```

## Interview Questions — Runners & Runtime

---

**Q: How does a self-hosted runner communicate with GitHub? What ports need to be open?**

A: The runner communicates outbound over HTTPS (port 443) using a long-polling protocol. No inbound ports are required. The runner polls GitHub's Actions service API for job messages, then streams logs back via WebSocket (also port 443). Only outbound 443 to github.com and \*.actions.githubusercontent.com is needed.

**Q: What is the difference between an artifact and a cache in GitHub Actions?**

A: Artifacts are output files from a workflow run (build binaries, test reports, logs) kept for a configurable retention period and available for download by users. Caches are dependency or build caches shared between runs of the same workflow to speed up builds. Caches expire after 7 days of non-access; artifacts after their retention period. Both are stored on GitHub infrastructure.

**Q: Why should ephemeral runners be used for self-hosted runners with public repositories?**

A: Persistent runners maintain state between jobs. A malicious PR can write secrets to disk, modify scripts in PATH, or leave backdoors that get picked up by subsequent legitimate jobs. Ephemeral runners start fresh for each job (a clean VM or container), eliminating this cross-job contamination risk.

**Q: Explain how ARC (Actions Runner Controller) auto-scales runners.**

A: ARC's HorizontalRunnerAutoscaler watches the GitHub Actions queue via the API (for org-level runners) or webhooks. When queued jobs exceed available runners, it scales up by creating new runner pods. When the queue drains, it scales down with a configurable cooldown period. The minimum scale-to-zero is supported for cost efficiency.